AgentWorkShop: A Modular Task-to-Device Integration Framework for Industrial Agents

***Abstract*—Industrial agents must connect tasks to scene-specific process signals and equipment interfaces while preserving evidence from operation requests to process responses. This paper presents AgentWorkShop, a modular task-to-device framework. It combines task and harness adapters, production context, named operations, semantic input/output (I/O), runtime checks, configured drivers, and service-owned records. Registered tasks and direct operation calls converge on shared dispatch while retaining route-specific records; the process experiments do not compare these entry paths. We evaluate four scenario-specific simulator missions in injection molding, anaerobic/anoxic/oxic (A2O) wastewater treatment, continuous annealing, and biaxially oriented polyethylene terephthalate (BOPET) film. The setpoints came from deterministic routines rather than language-model decisions; the logs retain process-variable (PV) observations and mission judgments. The last structured PV observations fell within the target bands for injection weight (32.370 g), BOPET thickness (25.30 μm), annealing hardness (98.14 HV), and wastewater dissolved oxygen (3.507 mg/L). The annealing mission summary reports 98.7 HV at 164 m/min, whereas the last structured hardness observation is 98.14 HV; final tensile and surface-defect readings are absent. The wastewater summary reports 3.55 mg/L dissolved oxygen and labels the effluent compliant, but the terminal effluent vector is absent and the reported value differs from the structured trace. Thus, the mission-level compliance flag cannot be independently verified. With one trajectory per scene, the results support configured execution in four preconfigured simulator scenes, not route equivalence, repeatability, arbitrary portability, model-selected optimization, or physical-plant performance.**

***Index Terms*—Industrial agents, industrial communication, industrial informatics, production systems, semantic data integration.**

I. Introduction

INDUSTRIAL-AGENT deployments must connect each task to its production context, process signals, and equipment interfaces. Prior work studies end-to-end automation, control-sequence generation, modular factory skills, and robotic-line control [**?**], [**?**], [**?**], [**?**], as well as programmable logic controller (PLC) code generation, context-aware control, resource discovery, shop-floor coordination, digital-twin fault handling, and natural-language machine access [**?**], [**?**], [**?**], [**?**], [**?**], [**?**]. Recent systems add Asset Administration Shell (AAS)-grounded OPC Unified Architecture (OPC UA) actions with signed approval, process and instrumentation diagram (P&ID)-grounded proposal checks, or governed sensor–manufacturing execution system (MES)–enterprise resource planning (ERP) evidence streams [**?**], [**?**], [**?**]. Across production lines, agent roles, signal maps, protocols, objectives, and product/run context vary. This paper addresses a software-integration question at that boundary: how can a task-to-device framework compose scene-specific mappings with shared operation handling while preserving evidence from invocation to process response? AgentWorkShop implements this architecture. Registered tasks and direct operation calls converge on named handlers. This is an implementation property, not a result tested by the process evaluation. We make no claim that equivalent behavior is absent from prior platforms.

The empirical question is narrower: can configured task workflows execute bounded operations and retain process observations and mission judgments in simulator scenes with different signals and target rules? We examine one deterministic control trajectory in each of four preconfigured scenes: injection molding, biaxially oriented polyethylene terephthalate (BOPET) film, continuous annealing, and anaerobic/anoxic/oxic (A2O) wastewater treatment. These runs assess configured execution, not automatic onboarding, repeatability, or transfer to unseen plants. Fig. **??** summarizes the design path: a task goal and active context inform an operation proposal; shared checks, route guards, configured approval, and a pre-dispatch context recheck govern writes through the device write-control (DCW) subsystem. A command receipt remains distinct from later process observations.

AgentWorkShop implements this path as a modular task-to-device framework. Registered harnesses connect collaborative tasks to production context, MES data, analysis and modeling services, DAQ and DCW nodes, and protocol-specific drivers. Command-line, web, software development kit (SDK), Representational State Transfer (REST), Model Context Protocol (MCP), and Agent-to-Agent (A2A) interfaces serve different callers. Task and chat requests enter the collaboration runtime; direct REST calls reach the shared dispatcher without creating

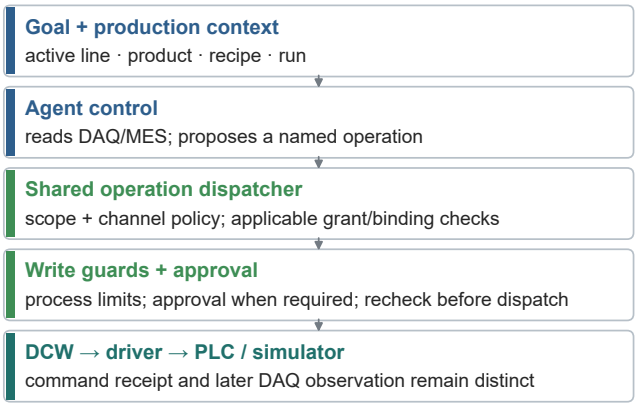


Fig. 1. Agent-control path for a configured industrial task. The agent uses data-acquisition (DAQ) and MES context, passes shared operation checks, and routes writes through process guards, conditional approval, and a pre-dispatch context recheck. DCW and configured drivers provide device access; command receipts and later process observations remain separate. The framework does not prescribe an optimization policy.

a task. Scene blueprints define signal maps, objectives, and guards, while channel profiles, tool policies, and agent bindings scope operations. The framework supplies this execution path without selecting routes autonomously or prescribing an optimization strategy. Fig. **??** shows the architecture, and Fig. **??** traces a task through it.

This paper makes two contributions. First, it presents a modular execution architecture combining scene configuration, harness adapters, named operations, shared checks, configured drivers, and service-owned records. For approved recipe actions, approval is bound to the active line–product–recipe–run tuple and rechecked before software dispatch; this check is not an atomic lock across device I/O. Second, it reports configured task-to-device execution in four simulator scenes through ordered operation requests, process observations, scenario-specific target bands, and mission-level judgments. The process results do not combine unlike objectives into one score or establish physical-plant performance.

Section II reviews related work. Section III presents the design rationale, architecture, and execution flow; Section IV describes module functions and integration. Section V reports the experimental protocol and results, Section VI discusses evidence limits, and Section VII concludes the paper.

II. Related Work

This section relates AgentWorkShop to plant-facing industrial agents and to production abstractions for context, tool access, and operational constraints. Table **??** is a focused functional comparison, not an exhaustive survey. It includes studies discussed below that report plant-facing actions, manufacturing task execution, or production-context and operation-evidence mechanisms. The table compares two aspects of the integration contract: whether task and direct-operation entries share a named handler, and how operational evidence is associated. Entries describe reported features; "not reported" does not imply absence, and evaluation descriptions retain each source's metrics.

A. Industrial Agents and Plant Integration

Industrial-agent systems, including large language model (LLM)-based systems, span end-to-end automation control, supervisory services, programmable logic controller (PLC) interfaces, manufacturing coordination, and operator-facing machine access. Xia et al. present an LLM agent framework for end-to-end industrial automation control, using structured prompts and event-driven context for production planning and operation [**?**]. Huang et al. present a multimodal architecture whose stated focus is automated control generation for industrial cyber-physical systems [**?**]. Biskupovic et al. study agent-based supervision for service-oriented industrial systems [**?**]; Agents4PLC and InstructMPC address PLC code generation and human–LLM context-aware control, respectively [**?**], [**?**]. Manufacturing studies consider dynamic resource-capability exploration [**?**], multi-agent shop-floor coordination [**?**], and digital-twin-supported fault handling [**?**]. Natural-language machine access [**?**] and real-time LLM control of a robot-operated serial line [**?**] extend this work to operator interfaces and robotic-line control. Together, these studies address different operational contexts, interfaces, and allocations of responsibility between agents, operators, and controllers.

The closest architecture-level comparison is Xia et al., who connect cognitive, digital-twin, and physical-automation layers and organize twin information as state snapshots, event messages, and plan sequences [**?**]. Other nearby designs expose factory-module operations through MCP with state-based tool filtering, constrain manufacturing plans, or compose semantic recipes from OPC UA skills [**?**], [**?**], [**?**]. Several recent systems also combine agent execution with approval, industrial data, or audit records. Graul et al. ground proposals in the Asset Administration Shell (AAS) and route approved actions through OPC UA Methods, with signed approvals and audit logging [**?**]. Schall separates advisory language-model agents from process and instrumentation diagram (P&ID)-grounded verification and execution, and evaluates structural checks across five simulated process scenarios [**?**]; its context includes supervisory control and data acquisition (SCADA) and standard operating procedure (SOP) information. Lin and Chen build a governed sensor–MES–ERP digital thread for closed-loop monitoring and control on real manufacturing records [**?**]. These works address asset-specific approval, process validation, or closed-loop analytics. The complementary design question is how a modular task-to-device framework can place registered-task and direct-operation entries behind shared named handlers while retaining route-specific records; the approved recipe path also rechecks the line–product–recipe–run tuple before software dispatch. This paper's process evaluation covers four configured simulator missions. It does not compare entry routes or report separate onboarding and context-guard experiments. Table **??** positions the implementation mechanisms and evaluation scope without implying comparative performance.

The operation checks complement safety-aware decision support [**?**] but do not replace independent process protection.

B. Production Context, Agent Interfaces, and Automation

Industrial integration also depends on abstractions that are not specific to agent architectures. The Model Context Protocol (MCP) specifies interoperable tool discovery and call mechanisms [**?**]; the operation-level scope, policy, grant, and binding checks discussed here are AgentWorkShop's own dispatcher rules, distinct from MCP transport authorization. In parallel, ISA-88 and ISA-95 describe recipes and production operations [**?**], [**?**]. Run-to-run methods connect process observations to successive production runs [**?**]. At the agent level, AgentSpec and CaMeL address tool behavior and capability-based information flow [**?**], [**?**]. Together, these standards and agent-level mechanisms provide abstractions for production context, tool interaction, and information flow. They motivate a focused design question: how can a modular task runtime and direct-operation interface converge on one named handler while preserving route-specific evidence under scene-specific signal and protocol mappings? AgentWorkShop implements this composition, and Section V reports process-execution trajectories from four configured simulator scenes. The evaluation does not test equivalence between task and direct-operation

TABLE I
REPORTED SHARED-ENTRY HANDLING AND EVIDENCE STRUCTURE IN SELECTED PLANT-FACING STUDIES. SHARED HANDLING MEANS THAT TASK AND DIRECT-OPERATION ENTRIES RESOLVE TO THE SAME NAMED HANDLER; "NOT REPORTED" IS NOT A CLAIM THAT A SYSTEM LACKS THE FEATURE. EVALUATIONS RETAIN SOURCE-SPECIFIC SCOPE AND CRITERIA.

System	Shared handler for task and direct entries	Reported operation and evidence structure	Reported evaluation
Xia et al. [**?**]	Not reported	Cognitive, twin, and physical layers; twin snapshots, event messages, and plan sequences	Robotic automation and process-simulation cases
Köhle et al. [**?**]	Not reported	Injected live state and state-based filtering; MCP-to-OPC UA operations and MQTT coordination	Nine simulated production challenges
Vieira da Silva et al. [**?**]	Not reported	Constrained multi-step plans over MCP manufacturing functions	Laboratory multi-step process
Köhle et al. [**?**]	Not reported	Semantic recipes composed from OPC UA skills; virtual commissioning	Unity assembly-workstation proof of concept
Graul et al. [**?**]	Not reported	AAS-grounded proposals, signed human approval, PLC-validated OPC UA Methods, and immutable audit log	Two laboratory PoCs; single scripted runs
Schall [**?**]	Not reported	P&ID-, SCADA-, and SOP-grounded proposal verification with gated execution	Five simulated scenarios; 43 proposals and 50 LLM runs
Lin and Chen [**?**]	Not reported	Governed sensor–MES–ERP digital thread with SOP-, risk-, and capability-aware action gating	25,275 real records; operational metrics
This work	Implemented; route equivalence not compared	Shared named dispatcher and route-specific record ownership are implementation properties	Four deterministic simulator missions with scenario-specific operation/PV sequences and judgments; no route comparison, onboarding, or guard ablation

entries, onboarding, or context-guard ablation, and it does not claim comparative platform performance or general plant portability.

III. SYSTEM DESIGN

AgentWorkShop is designed for industrial tasks whose agents, production contexts, interfaces, and process signals vary between deployments. Its architecture organizes these variations around shared task and operation services, checks each proposed operation against the active production context before device routing, and retains route-specific evidence that distinguishes a command receipt from a measured process response.

A. Design Rationale and Requirements

Five requirements—scene variability, harness variability, operation authority, outcome observability, and extensibility—shape the design. Scene variability is represented by blueprints and semantic driver mappings; harness variability by registered adapters; operation authority by shared dispatch checks and route-specific guards and admission; outcome observability by service-owned task, receipt, and observation records; and extensibility by registered interfaces for agent engines and device drivers. Tool discovery does not grant write authority, and actuator writes converge on the shared DCW admission point.

These mechanisms define an intended reuse boundary, not automatic portability. A new scene supplies a validated blueprint, policies, and driver mapping, while a new engine or protocol requires its configured adapter. The four simulator missions in Section **??** exercise configured scene workflows and retain process observations, but do not isolate dispatcher checks, compare records across entry routes, measure onboarding, substitute harnesses, or test driver extensibility. Those remain architecture-level claims in this study. Agent strategy and independent process protection remain outside the evaluation.

At setup, the integrator binds a blueprint to the active production tuple (line, product, recipe, and run), maps semantic process and control nodes to drivers, and declares target bands and route guards. At runtime, the dispatcher checks current authority before handing an accepted request to its route handler. The handler returns a command receipt, and DAQ records later process observations separately. The shared element is the execution contract; process semantics remain deployment-specific and require validation.

B. Layered Architecture

Fig. **??** summarizes the access routes, shared dispatch path, production services, semantic I/O, configured drivers, and service ownership of records. Task and chat routes enter the collaboration band, which contains workspaces, channels, task queues, shared memory, mailboxes, task templates, and harness adapters. A *task route* is an execution path through this runtime; a *harness adapter* connects an agent engine to its task execution interface. Direct REST tool calls and the stdio MCP operation bridge bypass task execution in that band (the upper-right direct-operation entry in Fig. **??**) and enter the shared governance block without creating a task or

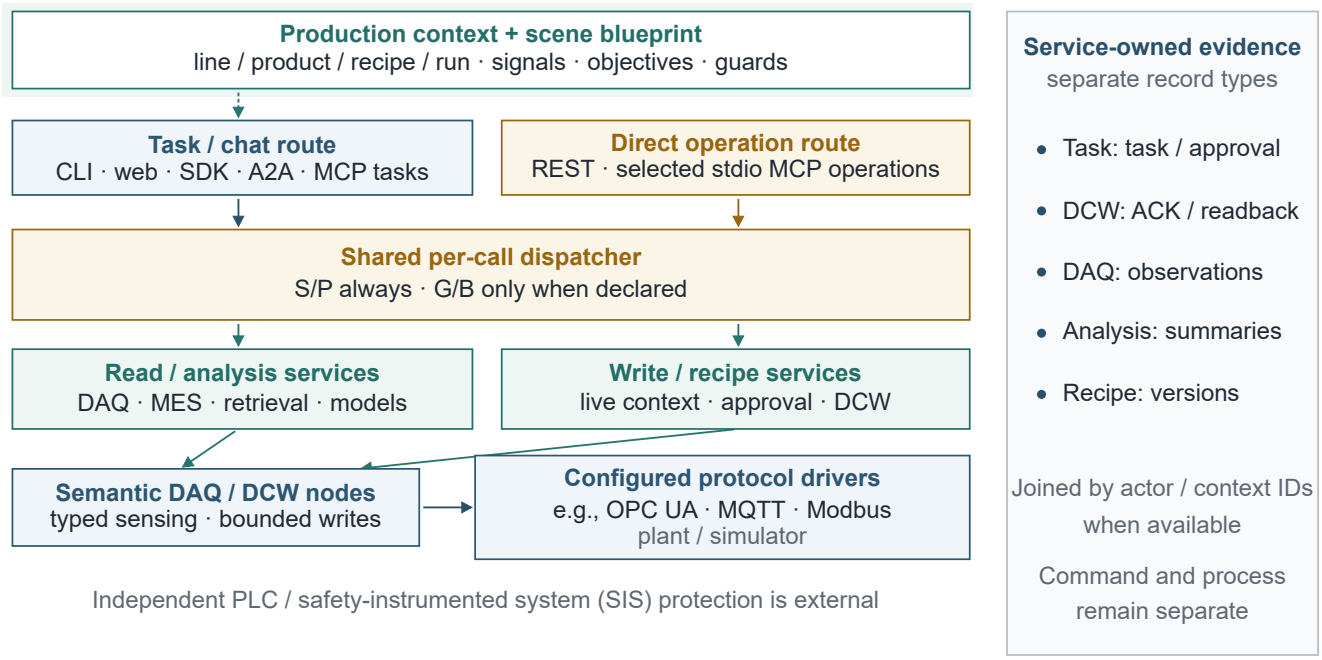


Fig. 2. Architecture and record ownership. Task/chat routes enter the collaboration runtime; direct REST and stdio-MCP calls enter shared dispatch checks without creating a task. One dashed link marks configuration. Write routes add approval and interval admission; recipe dispatch omits the previous-recipe window. The shared dispatcher always checks scope and channel policy, with grant and resource-binding checks applied when required by the operation. Owning services retain distinct records. Section V reports four configured simulator missions and does not compare entry routes. Independent PLC and safety-instrumented protection is external.

starting a harness turn; MCP management and task methods remain on their service routes. Direct calls resolve current route state at invocation, without an agent-turn prompt or a task record. The governance block projects the operation catalog and applies the per-call checks; write routes add the approval card and the admission interval. The context block supplies the active line–product–recipe–run tuple to the gate and scene maps, objectives, and guards to the handlers. The dashed link from that block in Fig. **??** marks this configuration relationship; the runtime assembles prompt-time context from the same records. The service band, behind a route-handler interface that selects an adapter, provides acquisition, write control, analysis, retrieval, and optional model services. Field I/O resolves semantic nodes to built-in or plugin drivers for a plant or simulator.

Each band has a distinct responsibility: task/chat routes enter the task runtime, direct operations enter the dispatcher, and both use shared operation checks. A harness adapter supplies agent-specific interaction, semantic nodes hide protocol details, and the driver registry provides the device-facing extension point. Owning services retain task, command, observation, and version records separately and link them to actor and production context; no single status conflates these outcomes.

C. Cross-Module Execution Flow

Fig. **??** summarizes a task turn from entry through operation execution, with route-specific evidence retained by owning services. Registered harness adapters receive a scoped tool catalog during setup and supported registry refreshes; the runtime assembles prompt-time production context for each agent turn. This context is grounding information, not a task-pinned transaction. For each returned tool call, the shared dispatcher rechecks invocation scope and channel policy and, for industrial operations, current grant and binding conditions before handler selection. The selected handler then applies route-specific guards, active-run checks, approval revalidation, and device-I/O steps as applicable. Command receipts and later process observations remain distinct records.

For an industrial operation u by agent a in channel c, define the shared dispatch condition as

$$\mathcal{A}(a,c,u) = \bigwedge_{X \in \mathcal{C}(u)} X(a,c,u), \tag{1}$$

where S, P, G, and B denote invocation scope, current channel policy, runtime grant, and the agent's binding to referenced resources, respectively. $\mathcal{C}(u) \subseteq \{S,P,G,B\}$ is the set of predicates declared for operation u; it always contains S and P. The grant predicate G is omitted only for an explicitly grant-exempt industrial tool, and B is included when the operation declares resource references. A declared no-reference operation omits B. Each included check must pass before handler selection. The policy predicate includes channel tool selection, profile, and duty-separation checks where configured. This condition is necessary for handler selection, not sufficient for actuation: the selected route applies its own guards and approval, then rechecks authority, context, and applicable limits before dispatch.

For direct REST invocation, the REST service establishes a and c before applying these checks. An agent token must match the requested agent identifier; otherwise, an authenti-

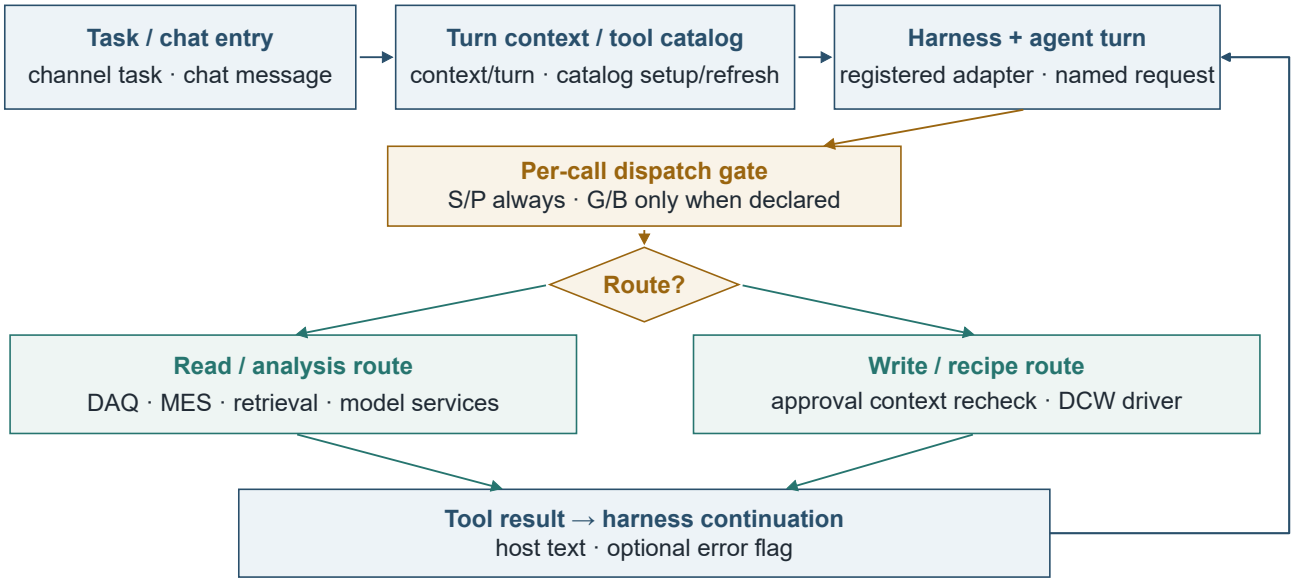


Fig. 3. Registered-harness task flow. The runtime invokes the adapter with a scoped catalog. Scope and channel policy are checked for every call; grant applies to non-exempt industrial operations, and binding applies when resource references are declared (figure label: S/P always, G/B only when declared). Failed checks return tool errors. The read path retrieves data. The write path applies route guards, approval and context revalidation, DCW admission, and receipt grading; a later process observation is separate. Results return to the harness. Checks occur at the software boundary and reserve no equipment state.

cated user must be the channel owner or an administrator. The manager loads the channel-agent record to obtain the agent identity, role, and channel, then rechecks invocation scope and channel tool policy. Industrial operations continue through the declared shared grant and binding checks. Handlers resolve live production state from request identifiers and owning services, supplying the authorization and production context for direct dispatch.

Fig. **??** gives reference pseudocode for the registered-harness task path, not one literal orchestrator function. Direct REST tool invocation bypasses the task runtime and harness but enters the same shared dispatcher. Route-specific timing, approval, and evidence rules are detailed in Section **??**.

IV. Module Functions and Integration

The design in Section **??** is realized through three cooperating functions: task and context assembly, shared operation dispatch, and controlled device access with route-specific evidence. We follow one agent action through them.

A. Task, Harness, and Context

Task and chat entry points create or join a workspace channel. The task runtime manages membership, task state, and shared execution context; a harness registry adapts each supported agent engine to a common execution interface. At each agent turn, the prompt builder assembles the task, active scene description, production context, channel profile, agent binding, and retrieved memory. This keeps engine-specific interaction behind the harness interface while preserving the current line and operation context for the agent.

Require: task q, channel c, bound agent a, registered scoped catalog T
Ensure: task result; any retained evidence remains with its owning service

$x_t \leftarrow$ ResolvePromptContext(c, q)
$m \leftarrow$ RetrieveMemory(q)
$p \leftarrow$ AssemblePrompt(q, x_t, m)
$h \leftarrow$ InvokeRegisteredHarness(a, p, T)
for all tool request u returned by h **do**
  $g \leftarrow$ CheckScopeAndChannelPolicy(a, c, u)
  if not g **then**
    EmitToolErrorToHarness(h,u); **continue**
  end if
  if u is an industrial operation **then**
    $v \leftarrow$ CheckApplicableGrantBinding(a, u)
    if not v **then**
      EmitToolErrorToHarness(h,u)
      continue
    end if
  end if
  $k \leftarrow$ SelectHandler(u)
  $r \leftarrow$ ExecuteRouteHandler(k, a, c, u) ▷ handler resolves live state and may persist route-specific records
  EmitDeclaredToolResultToHarness(h,r)
end for
return task state and tool results

Fig. 4. Reference pseudocode for the registered-harness task path. Direct operation invocation is not shown. Evidence persistence is route-specific and remains with the owning service; the shared tool result does not carry a generic evidence envelope.

Direct REST and selected stdio-MCP operation calls reach

the same dispatcher without a task or harness turn. REST authentication and identity resolution follow Section **??**; each handler reads live route state at invocation. Handlers do not branch on the entry route: they receive the resolved actor, channel, and operation arguments, and the records they write are keyed by the identifiers that request supplies. For either entry path, prompt context and tool-catalog visibility describe available information and capabilities; neither grants write authority.

B. Shared Dispatch and Device Admission

The dispatcher is the shared operation boundary. It checks invocation scope and current channel policy, then verifies the agent binding, runtime grant, and ownership of referenced resources before selecting a handler. In the notation of (**??**), scope and policy apply to every call; grant is skipped only for a grant-exempt tool, and binding applies only when references are declared. Handlers revalidate route-specific state at execution time; a previously displayed tool catalog does not preserve authority after a policy or grant changes. Scene blueprints define device and signal maps, objectives, and process guards. Capability profiles and tool policies narrow available operations, while bindings constrain accessible nodes, controls, and recipes. Optional DAQ, MES, analysis, retrieval, optimization, and digital-twin services are invoked through registered handlers; the framework does not prescribe a single optimization strategy.

Actuator writes converge on the shared DCW admission point. For an online node write, the admissible value interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \tag{2}$$

where $\mathcal{H}_n$ is the hard engineering range and $\mathcal{B}_n$, $\mathcal{P}_n$, and $\mathcal{R}_n$ are the baseline, active-product, and active-recipe limits; the subscripted limits here are distinct from the policy and binding predicates of (**??**). Missing optional limits impose no restriction; an empty intersection rejects the request and identifies the excluding layer. Recipe dispatch validates the replacement recipe against the hard, baseline, and product limits rather than the previous recipe window. Step, interval, and hold-window checks also constrain each route. These software checks bound commands but do not replace independent programmable-logic-controller or safety-instrumented protection.

C. Approval, Execution, and Evidence

Routes add timing and approval checks appropriate to their operation. Online node writes enforce a 60 s minimum spacing; recipe trials use a per-line cadence (300 s by default), and recipe intervals are anchored to approval. Manual approval binds an actor and proposed operation to candidate values, applicable limits, expiry, and the active line–product–recipe–run tuple. After approval, the handler rechecks authority and compares the current tuple with the approved tuple before entering DCW dispatch. The default approval wait expires after 180 s. A changed recipe definition requires a new `recipe_apply` proposal. These checks reject stale requests at the software dispatch boundary; they do not lock equipment state across network or device I/O.

Fig. **??**(a) illustrates the four-layer value interval in Eq. (**??**) using an illustrative rotational-speed configuration in revolutions per minute (rpm). Panel (b) shows a symbolic approval followed by authority and context rechecks; its values are illustrative, not experimental measurements.

A `recipe_trial` illustrates the operation sequence: the handler checks the bound recipe, active run, candidate parameters, and route timing; if required, it requests approval; after revalidation, it submits a bounded batch through the configured driver and returns a per-parameter receipt. The caller obtains later DAQ observations separately and may then update, apply, or roll back a recipe. A trial neither saves nor applies a recipe version. The driver registry maps semantic controls to configured protocol drivers, keeping protocol details out of agent operation arguments.

Receipts and observations answer different questions. A command result is graded as readback-verified, transport-acknowledged, or unverified; a readback mismatch can trigger up to three reads at 2 s intervals. Recipe batches retain per-parameter verification outcomes. DAQ stores subsequent timestamped process observations, while recipe versions, approvals, and task records remain service-owned records linked where a route supplies the corresponding identifiers. A verified setpoint confirms the command path, not target attainment. This distinction defines the evidence chain evaluated in Section **??**.

V. Experimental Evaluation

This evaluation asks whether a configured task-to-device workflow can execute scenario-specific control sequences, retain process-response evidence, and judge the stated target in heterogeneous industrial simulator scenes. The reported data come from one integrated benchmark output containing four scenario-specific missions: injection molding, A2O wastewater treatment, continuous annealing, and biaxially oriented polyethylene terephthalate (BOPET) film. The BOPET mission was provisioned in a separate sub-run and combined with the other three mission logs in the same benchmark output. Here, a scene denotes a configured process setting, and a mission denotes the single trajectory executed in that scene. The four trajectories are not repeated trials of a common objective.

A. Test Design and Procedure

Each scene used its own configured process-variable (PV) target, control variables, signal map, and simulator preset. The task execution shell used a mock lead/worker team and a deterministic, non-LLM control routine. It read the initial PV through data-acquisition (DAQ) before any operation, issued bounded setpoint requests through the configured platform operation interface, waited for the simulator response, and recorded the resulting PV and operation judgment. A candidate was retained or restored according to the scenario rule. Each mission is one ordered trace; its individual writes are sequential steps, not independent process trials. The experiment therefore tests request, response, and outcome recording for configured scenes. It does not test model-selected setpoints,

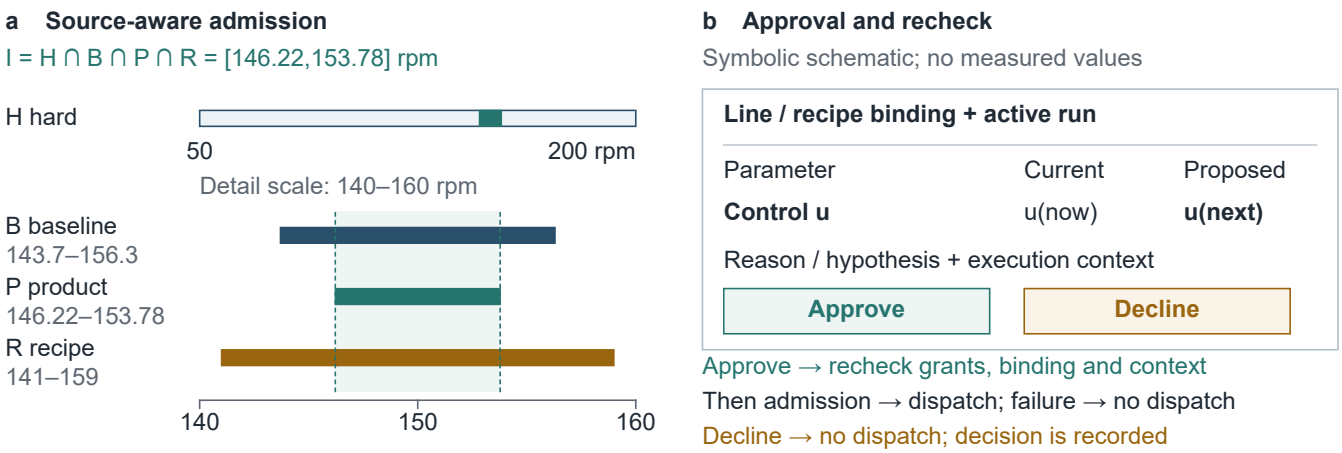


Fig. 5. Admission and approval boundaries. (a) Illustrative rotational-speed configuration in revolutions per minute (rpm), not experimental data: online node interventions use the four-layer interval of Eq. (**??**), whereas recipe dispatch omits the previous recipe window. (b) Symbolic human approval binds a proposed action and its context tuple; execution rechecks the active line–product–recipe–run context, authority, and applicable limits. Approval does not reserve equipment state.

compare entry routes, or evaluate onboarding, harness substitution, or context-guard behavior.

The primary reported outcome for each mission was its scenario-specific mission judgment under the target and guard rules. We assessed trace-level target attainment from structured PV observations and available secondary readings. The initial PV sample is a within-mission reference, not a competing controller or platform baseline. Fig. **??** plots the logged PV sequence in native units and its target band. Event order, rather than elapsed time, is used on the horizontal axis because the mission traces lack consistent timestamps; the plots therefore do not establish response latency or settling time. The table and figure use the same four mission logs and retain their separate objectives. The wastewater analysis uses the structured dissolved-oxygen (DO) target (3.4±0.6 mg/L), which drives the recorded target-band check. The free-text description states 3.2±0.6 mg/L, and the source records do not resolve the intended value. No physical PLC or production line was tested.

Three evidence types remain separate in the analysis: the process-variable trajectory, secondary constraint readings, and the mission's terminal report. Only recorded PV samples form the plotted sequence; a separately reported endpoint remains distinct when it differs from that sequence. We report secondary constraints only when present in the logs and do not infer missing values from a mission-level compliance label. This distinction separates successful command handling from evidence that a process target was attained.

B. Process Trajectories and Results

Fig. **??** shows the four PV trajectories against their own target bands. The plotted markers are recorded observations, not interpolated or averaged values. The open marker in the annealing panel identifies the observation after the zone-3 request that lacks a linked judgment record. Separate squares show the annealing and wastewater mission-end values; they are not connected to the structured trajectories because the logs report different terminal values. The wastewater panel plots the aeration-stage DO sequence; later pH and total-phosphorus actions use different PVs and are summarized in Table **??**.

In injection molding, the three judged operations moved part weight from 31.262 to 32.370 g, ending inside the 32.15–32.85 g target band. The mission log also reports flash at 0.0158% (limit 0.4%) and sink index 0.9458 (limit 1.5). In BOPET film, four keep judgments across three control variables reduced thickness from 28.00 to 25.30 μm, inside the 24.3–25.7 μm band; the companion log records melt temperature within its configured interval. These two traces provide the clearest record-linked examples of the operation–response–judgment sequence.

The annealing mission first brought the logged hardness observations into the 95 ±6 HV band, then increased line speed from 140 to 164 m/min while the structured hardness values remained below the 100 HV line-speed margin cutoff. The narrative log reports 100.8 HV at the reverted 176 m/min candidate. Although this is within the 89–101 HV target band, it exceeds the stricter margin cutoff. The initial temperature action has no linked optimization record, and final tensile-strength and surface-defect readings are not retained. We therefore report the mission outcome without treating the trace as an independent verification of every quality constraint.

In wastewater treatment, the structured DO observations entered the 2.8–4.0 mg/L band after the second blower adjustment. The mission then addressed pH and total phosphorus using separate NaOH and PAC stages; the records report pH 6.62 and TP decreasing from 0.549 to 0.416 mg/L against a 0.5 mg/L upper limit. The simulator defines its cost index as $J = 100(f/30)^{2.6} + 0.30n + 0.45p$, where f is blower frequency and n and p are the NaOH and PAC setpoints. The reported index changed from 181.2 to 180.0; this modeled value is not measured energy or monetary cost. The final effluent vector is absent, and the mission-reported DO endpoint differs from the last structured observation, so the compliance flag is not an independently verified terminal effluent assessment.

The missions probe different configuration demands: mul-

TABLE II
SCENARIO OBJECTIVES, CONTROL SEQUENCES, AND MISSION-REPORTED OUTCOMES IN THE INTEGRATED SIMULATOR BENCHMARK. NUMERICAL VALUES RETAIN SOURCE-LOG PRECISION.

Scenario and target	Process trajectory and operations	Reported outcome and evidence boundary
Injection molding Weight: 32.5 ± 0.35 g	Three record-linked, judged actions changed hold pressure, hold time, and hold pressure again. Weight progressed from 31.262 to 31.913, 32.140, and 32.370 g.	The terminal weight is within [32.15, 32.85] g. The log reports flash rate 0.0158% (limit 0.4%) and sink index 0.9458 (limit 1.5).
BOPET film Thickness: 25.0 ± 0.7 μm	Four record-linked, judged actions adjusted casting speed, fast-roll speed, and rail-out width. Thickness decreased from 28.00 to 26.58, 25.98, 25.78, and 25.30 μm.	The terminal thickness is within [24.3, 25.7] μm. The companion execution log reports melt temperature of 291.5–291.8 °C within the configured 268–300 °C interval and 9/9 driver checks.
Continuous annealing Hardness: 95 ± 6 HV; throughput: $\geq 5\%$	The hardness trajectory decreased from 134.92 to 99.38 HV after an unlinked zone-3 action, then to 99.42 and 98.14 HV after two zone-2 trims. Two speed increases took the line from 140 to 164 m/min; a 176 m/min candidate was reverted.	The mission log reports a final hardness of 98.7 HV in [89, 101] HV and a 17.14% speed increase, above the 5% mission threshold. Its narrative records the reverted 176 m/min candidate at 100.8 HV: below the 101 HV target limit but above the 100 HV margin cutoff used for speed acceptance. This value is absent from the structured trajectory. The first action lacks a linked judgment record, and final tensile and surface-defect values are absent.
A2O wastewater Dissolved oxygen (DO): 3.4 ± 0.6 mg/L	Five blower adjustments increased structured dissolved oxygen (DO) from 0.228 to 2.157, 2.807, 2.860, 3.227, and 3.507 mg/L. A NaOH adjustment reported pH 6.62 (configured range 6–9); three polyaluminium-chloride (PAC) actions reduced total phosphorus (TP) from 0.549 to 0.416 mg/L (configured upper limit 0.5 mg/L). Three later cost-trim writes were restored after checks.	The mission summary reports DO of 3.55 mg/L and labels the effluent compliant. Its simulator cost index changed from 181.2 to 180.0; this is not measured energy or currency. The outcome DO differs from the final structured trace value, and a complete terminal effluent vector is not archived.

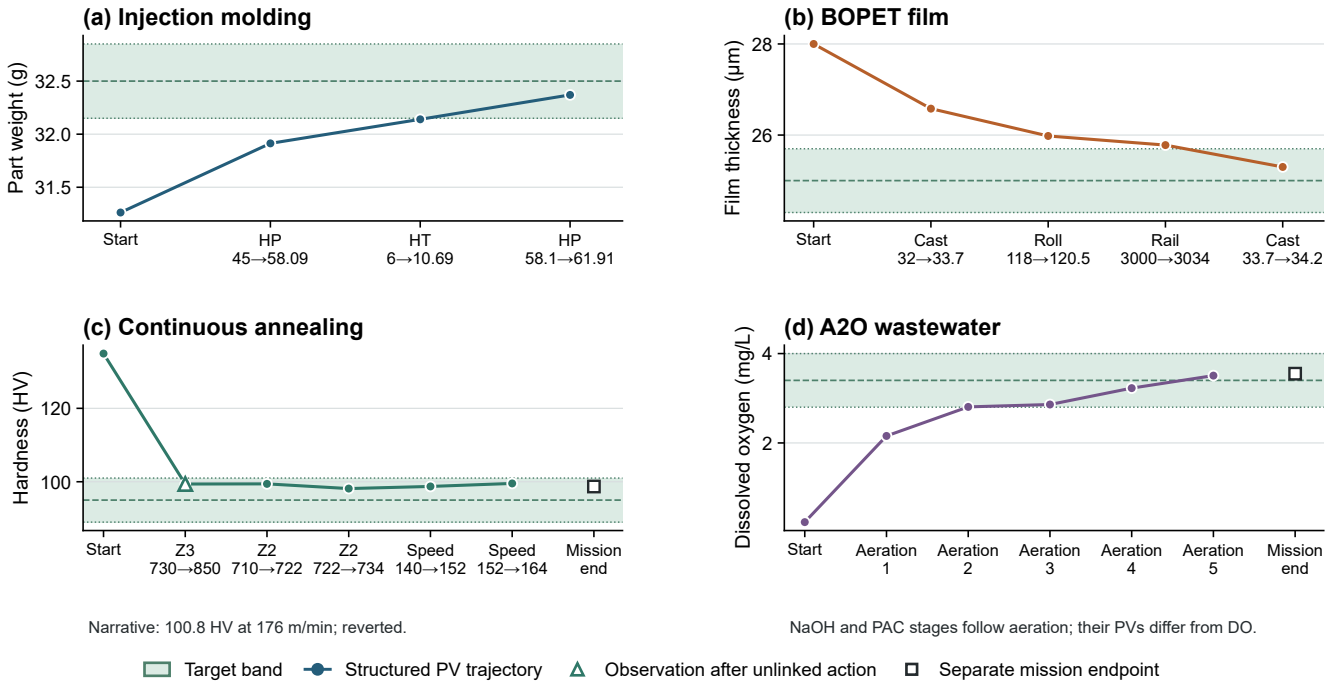


Fig. 6. Four scenario-specific process trajectories from the integrated simulator benchmark output. Each panel uses its native process-variable unit; the shaded region marks that scenario's target band, and the horizontal axis shows event order rather than elapsed time. HP/HT denote hold pressure/time; Z2/Z3 denote annealing zones. Circles mark structured observations, the open triangle marks the annealing observation after an action without a linked judgment record, and open squares mark separately reported mission endpoints that are not connected to the structured trajectory. The wastewater sequence plots dissolved oxygen during aeration; later NaOH and PAC stages act on different variables. The reverted 176 m/min annealing candidate appears only in the narrative log as 100.8 HV, above the 100 HV margin cutoff, and is not plotted as a structured process observation.

tiple related setpoints in injection molding, three actuator nodes in BOPET, a hardness–throughput trade-off in annealing, and sequential aeration and chemical-treatment stages in A2O. This variation illustrates what remains common at the framework boundary while each control problem stays scene-specific. It does not test transfer of a shared control law.

C. Interpretation and Limitations

The records support configured task-to-device execution across four simulator profiles, each with its own signal map, controls, and target rules. Evidence completeness differs: injection and BOPET link judged operations to their reported endpoints, whereas annealing and wastewater have incomplete or discordant terminal records. The deterministic routines and unlike objectives do not establish learned optimization, arbitrary-scene portability, or comparative performance.

Each scenario has one trajectory and no independent process replication, so the study provides no estimate of run-to-run variation or statistical uncertainty. The archived records and plotting script support reconstruction of the displayed trajectories, but not exact mission replay: the bundle lacks a run manifest, source revision, and consistent timing metadata. All observations come from local simulators, not physical equipment; the findings are bounded to execution in these four preconfigured scenes.

VI. Discussion

A. Interpretation

The architecture separates shared task and dispatch services from scene-specific mappings and target rules. The four missions illustrate configured execution under this separation, but do not measure configuration effort or transfer to unseen scenes. Shared dispatch and route-specific record ownership remain implementation properties; the experiments do not establish route equivalence or comparative benefit. This design complements prior work on task-level industrial agents and plant-specific action checks by making the task-to-device boundary explicit [**?**], [**?**], [**?**].

The incomplete annealing and wastewater endpoints also have a system-level implication. A mission-level success flag cannot verify terminal quality when the final process vector is missing or differs from the structured trace. A deployment that relies on process evidence should declare required terminal variables for each scene and persist their observations with operation records. This is a design requirement suggested by the evidence gaps, not a feature validated by this benchmark.

The process objectives also remain distinct. Part weight, film thickness, hardness, dissolved oxygen, and the simulator-defined wastewater cost index have different units and meanings; they cannot be pooled into one score. Because the routines are deterministic and do not use a language model, the study evaluates configured execution rather than learned reasoning or optimization-policy quality.

B. Scope and Limitations

One deterministic trajectory per scene, without an alternative policy, precludes claims about repeatability or comparative process benefit. A paired test of identical operations through task and direct entry would assess shared handler behavior and route-specific records; a new-scene onboarding study would measure configuration effort and reuse. The local simulator results do not establish physical transfer, operator effectiveness, or energy and chemical savings. Hardware evaluation would require synchronized terminal guard records and independent PLC or safety-instrumented protection.

VII. Conclusion

AgentWorkShop provides a modular task-to-device framework that connects configured scene context, named operations, device drivers, and service-owned records. One integrated simulator benchmark output contains four scenario-specific missions with ordered operations, structured process observations, and mission judgments. Structured observations entered the scenario target bands, while the annealing quality readings and wastewater terminal effluent vector remain incomplete or discordant. The wastewater mission summary reports compliance, but the archived evidence does not independently verify the terminal effluent state. The results support configured execution in these four preconfigured simulator scenes, not route equivalence, automatic onboarding, model-selected optimization, repeatability, comparative process improvement, general portability, or physical-plant performance.

References

[1] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language model agents," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205539.

[2] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[3] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026, pp. 1–8, doi: 10.1109/ETFA68377.2026.11725257.

[4] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," *Mach. Learn.: Eng.*, vol. 2, no. 1, 2026, art. no. 015006; doi: 10.1088/3049-4761/ae4ee8.

[5] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[6] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. IEEE 64th Conf. on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[7] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[8] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[9] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205597.

[10] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[11] C. Graul, W. Żarski, D. Mikołajewski, and I. Rojek, "Ai agents in industry 4.0: AAS–OPC UA–LLM architecture as the foundation of intelligent manufacturing systems in the context of industrial enterprise implementation," *Applied Sciences*, vol. 16, no. 11, p. 5428, 2026, doi: 10.3390/app16115428.

[12] D. Schall, "Safe integration of large language models into industrial process control: A multi-agent architecture with P&ID-grounded validation," *Autonomous Intelligent Systems*, vol. 6, 2026, article no. 14; doi: 10.1007/s43684-026-00136-1.

[13] S.-C. Lin and C.-Y. Chen, "An integrated framework featuring policy-governed agentic AI for closed-loop manufacturing control with multi-source sensor–MES–ERP," *The International Journal of Advanced Manufacturing Technology*, vol. 143, pp. 6405–6428, 2026, doi: 10.1007/s00170-026-17806-2.

[14] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[15] Y. Xia, N. Jazdi, and M. Weyrich, "An architecture for integrating large language models with digital twins and automation systems," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205636.

[16] L. M. Vieira da Silva, A. Köcher, and F. Gehlhoff, "Beyond formal semantics for capabilities and skills: Model context protocol in manufacturing," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–4, doi: 10.1109/ETFA65518.2025.11205601.

[17] K. Köhle, D. Anicic, T. A. Runkler, K. Dorofeev, A. S. Thuluva, and R. Graf, "Virtually commissioned semantic recipes based on OPC UA skills," in *Proc. IEEE 8th Int. Conf. on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–6, doi: 10.1109/ICPS65515.2025.11087832.

[18] X. Jiang *et al.*, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," *Processes*, vol. 14, no. 13, 2026, art. no. 2112; doi: 10.3390/pr14132112.

[19] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, version 2026-07-28, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[20] ISA, "ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[21] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[22] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2000.

[23] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. on Software Engineering (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[24] E. Debenedetti *et al.*, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813. [Online]. Available: https://arxiv.org/abs/2503.18813